TRAKYA ÜNİVERSİTESİ · BİLGİSAYAR MÜHENDİSLİĞİ

Dosya Organizasyonu

Hafta 6 · Dosya Sıkıştırma

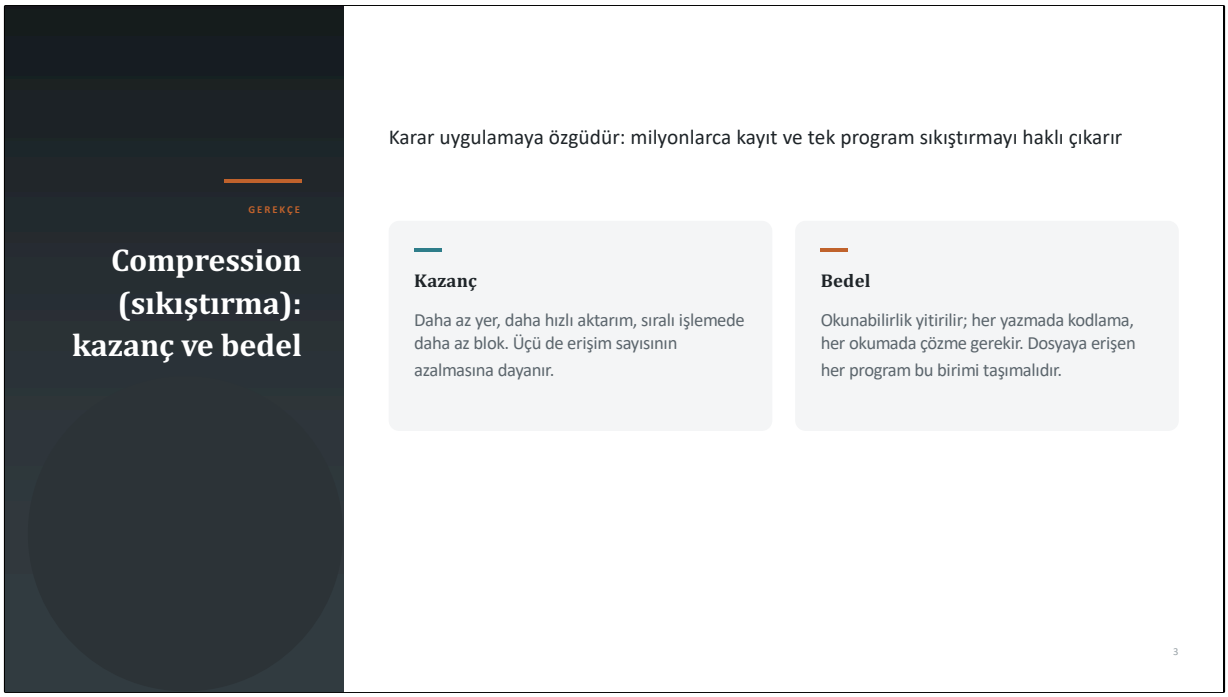
Doç. Dr. Emir Öztürk

2025/2026 Güz Yarıyılı

1

Gerekçe ve compact notation

Kazanç, bedel ve derli toplu gösterimle fazlalığın atılması



Compression (sıkıştırma) dosyayı daha az yer kaplar duruma getirir, daha hızlı aktarılmasını sağlar ve sıralı işlemede okunacak blok sayısını azaltır. Üç kazancın kaynağı aynıdır: erişim sayısının azalması. Bedeli de üç başlıkta toplanır: ikili kodlanmış dosya metin düzenleyiciyle açılamaz, her yazmada kodlama ve her okumada çözme için işlem zamanı harcanır ve dosyaya erişen her program kodlama ile çözme birimini taşımak zorundadır.

ÖRNEK

İl kodu alanı

Gösterim	Yer	Taşıdığı bilgi
İki ASCII karakter	16 bit	81 değer
İkili sayı olarak	7 bit	81 değer
Tek bayta yerleştirilmiş	8 bit	81 değer

Kayıt başına 1 bayt kazanç: yüzde elli

– Atılan 9 bit hiçbir bilgi taşımıyordu

Compact notation (derli toplu gösterim): geri döndürülebilir fazlalık azaltma

Değer kümesi küçükse değerler numaralandırılır, numara saklanır

4

Bir kayıtta il plaka kodunu tutan alan, “22” gibi iki ASCII karakterle yazıldığında on altı bit yer kaplamaktadır. Alanın alabileceği değer sayısı ise seksen bir ile sınırlıdır ve bunları ayırt etmek için yedi bit yeterlidir. Alan tek bayta sığdırıldığında her kayıtta yüzde elli tasarruf sağlanır. Atılan dokuz bit fazlalıktı, çünkü hiçbir bilgi taşımıyordu. Bu yaklaşım compact notation olarak adlandırılır ve geri döndürülebilirdir: çözüldüğünde özgün veri eksiksiz elde edilir. Fixed-length field (sabit uzunluklu alan) en uzun değere göre ayrıldığından değer kümesi sonlu ve küçük olduğunda değerleri numaralandırıp numarayı saklamak her kayıtta yinelenen bir kazanç üretir.

UYGULAMA

Gereken en az bit sayısı

Alan	Değer sayısı	En az bit	Metin olarak
İl kodu	81	7 bit	2 bayt
Ay	12	4 bit	2 bayt
Gün	31	5 bit	2 bayt
Yıl (1900 ile 2155)	256	8 bit	4 bayt
Tarih: üçü birlikte	:	17 bit → 3 bayt	8 bayt

Bedeli: alan artık doğrudan okunamaz

Kodlama ve çözme, alana erişen her programda bulunmalıdır

5

Tabloda birkaç alan için gereken en az bit sayısı hesaplanmıştır. İl kodu için seksen bir değer yedi bitle, ay bilgisi on iki değer için dört bitle, gün otuz bir değer için beş bitle gösterilebilir. Tarih alanı ayrı ayrı kodlandığında iki bayta sığmaktadır. Aynı bilgi sekiz karakterlik metin olarak sekiz bayt tutar. Kodlamanın bedeli, alanın artık doğrudan okunamamasıdır.

2

Tekrar eden dizilerin küçültülmesi

Run-length encoding (yineleme uzunluğu kodlaması)

ALGORITMA

Escape byte (kaçış baytlı) kodlama

Veride geçmediği bilinen bir bayt kaçış baytı seçilir

Dizi baştan sona okunur, değerler olduğu gibi kopyalanır

Aynı değer art arda birden çok kez geçtiğinde üç bayt yazılır

- Kaçış baytı
- Yinelenen değer
- Yineleme sayısı

Sayı tek bayta yazıldığından bir kodlama en çok 256 yinelemeyi karşılar

Algoritma önce, veride geçmediği bilinen bir bayt değeri escape byte (kaçış baytı) olarak seçer. Ardından dizi baştan sona okunur ve değerler olduğu gibi kopyalanır. Aynı değer art arda birden çok kez geçtiğinde, bu değerlerin yerine üç bayt yazılır: kaçış baytı, yinelenen değer ve yineleme sayısı. Sayı tek bayta yazıldığı için bir kodlama en çok iki yüz elli altı yinelemeyi karşılar. Daha uzun koşular birden çok kodlamaya bölünür.

ÖRNEK

18 bayt → 11 bayt



Kazanç yüzde otuz dokuz

Çözme tek geçişte yapılır: escape byte (kaçış baytı) görüldüğünde iki bayt okunur

8

Örnekte on sekiz baytlık dizi on bir bayta inmektedir. İlk üç değer olduğu gibi kopyalanır. Yedi kez yinelenen değer üç bayta, altı kez yinelenen değer yine üç bayta indirilir. Kalan iki değer kopyalanır. Kazanç yüzde otuz dokuz dolayındadır. Kodlamanın çözülmesi tek geçişte yapılır: escape byte (kaçış baytı) görüldüğünde izleyen iki bayt okunur ve değer belirtilen sayıda yinelenerek yazılır.

3

Variable-length code (değişken uzunluklu kodlar)

Sık geçen değere kısa kod

YÖNTEM

Huffman kodlaması

- Her değerin geçme olasılığı belirlenir
- Olasılıklardan bir ikili ağaç kurulur
- Değere giden yol, o değerin kodudur
  - Sık geçen değerlere kısa yollar düşer
- Ağaç bir tabloya dönüştürülür, kodlama ve çözme tablo üzerinden yapılır
- Tablo veriden üretildiği için dağılımı bilinmeyen veride de çalışır

Huffman kodlaması, veri kümesindeki her değerin geçme olasılığını belirler ve bu olasılıklardan bir ikili ağaç kurar. Ağaçta her değere giden yol, o değerin kodudur. Sık geçen değerlere kısa yollar düşer. Kurulan ağaç bir tabloya dönüştürülür ve kodlama ile çözme bu tablo üzerinden yapılır. Yöntemin belirleyici özelliği, tablonun veriye göre üretilmesi ve bu nedenle dağılımı önceden bilinmeyen verilerde de çalışmasıdır.

ÖRNEK

Yedi harf için kod tablosu

Harf	a	b	c	d	e	f	g
Olasılık	0,4	0,1	0,1	0,1	0,1	0,1	0,1
Kod	1	010	011	0000	0001	0010	0011
Bit	1	3	3	4	4	4	4

Yedi değer için sabit uzunlukta 3 bit yeterli olurdu

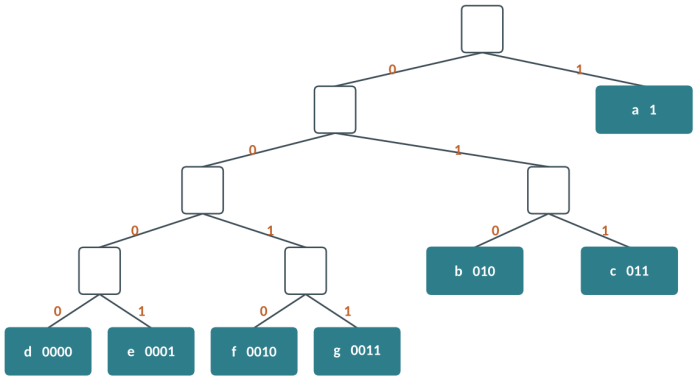
Burada dört bitlik kodlar da var

- Kodların ayraçsız ayırt edilebilmesi için ödenen bedel

“abde” → 1 010 0000 0001

YAPI

Huffman ağacı



Değer yalnızca leaf (yapraklara) atanır

Ağaçta her iç node (düğümün) iki dalı vardır ve dallar sıfır ile bir bitine karşılık gelir. Root (kökten) bir yaprağa giden yol boyunca okunan bitler, o yaprağın kodunu verir. En sık geçen değer köke en yakın yaprağa yerleştirildiği için en kısa kodu alır. Yalnızca leaf (yapraklara) değer atanması, hiçbir kodun bir başkasının başlangıcı olamayacağını güvence altına alır.

HESAP

Ortalama kod uzunluğu

$$0{,}4 \times 1 + 0{,}2 \times 3 + 0{,}4 \times 4 = 2{,}6 \text{ bit}$$

Sabit uzunluklu gösterim: 3 bit

Kazanç yüzde on üç dolayındadır

Kazancın büyüklüğü dağılımın dengesizliğine bağlıdır

- Bütün değerler eşit sıklıktaysa Huffman bir üstünlük sağlamaz

En büyük kazanç, birkaç değerin baskın olduğu dağılımlarda elde edilir

Ortalama kod uzunluğu, her değerin kod uzunluğunun olasılığıyla çarpılıp toplanmasıyla bulunur. Örnekteki dağılım için bu değer 2,6 bit çıkmaktadır. Aynı yedi değerin sabit uzunlukla gösterilmesi 3 bit gerektirir. Kazanç yüzde on üç dolayındadır. Kazancın büyüklüğü dağılımın ne kadar dengesiz olduğuna bağlıdır. Bütün değerler eşit sıklıkta geçtiğinde Huffman kodlaması sabit uzunluklu kodlamaya göre bir üstünlük sağlamaz.

ÖZELLİK

Prefix code (önek kodu)

Hiçbir kod, bir başka kodun başlangıcı değildir

Bu nedenle kodlar ayraçsız art arda yazılabilir

Çözme: bitler soldan sağa okunur, ağaçta ilerlenir

- Yaprağa varılınca değer yazılır, root (köke) dönülür

Dördüncü haftadaki alan ayracının karşılığı burada ağacın kendisidir

Kodların ayraç kullanılmadan art arda yazılabilmesi, hiçbir kodun bir başka kodun başlangıcı olmamasıyla sağlanır. Bu özelliğe prefix code (önek kodu) denir. Çözme sırasında bitler soldan sağa okunur ve ağaçta ilerlenir. Bir yaprağa varıldığında o değer yazılır ve köke dönülür. Bu nedenle kodlanmış akımda kodların nerede bittiğini gösteren ek bir işarete gereksinim duyulmaz.

YÖNTEM

Ağacın kurulması

Ağaç aşağıdan yukarıya kurulur

Her adımda en küçük iki olasılık birleştirilir

Adım	Birleştirilen	Yeni düğüm	Kalan liste
1	b 0,1 + c 0,1	0,2	a · d · e · f · g · 0,2
2	d 0,1 + e 0,1	0,2	a · f · g · 0,2 · 0,2
3	f 0,1 + g 0,1	0,2	a · 0,2 · 0,2 · 0,2
4	0,2 + 0,2	0,4	a 0,4 · 0,2 · 0,4
5	0,2 + 0,4	0,6	a 0,4 · 0,6
6	0,4 + 0,6	1,0	kök

En küçük olasılıklar en son birleşir: köke en uzak yapraklara düşerler

15

Ağaç aşağıdan yukarıya kurulur. Her adımda en küçük iki olasılık seçilir ve bunlar, olasılıkları toplanarak tek bir düğümde birleştirilir. Birleştirilen düğüm listeye geri konur ve işlem tek düğüm kalana dek sürer. Son kalan düğüm ağacın köküdür. En küçük olasılıkların en son birleştirilmesi, onların köke en uzak yapraklara düşmesini ve dolayısıyla en uzun kodları almasını sağlar.

ADIM 0 / 6

Huffman ağacı: başlangıç

Başlangıç: yedi yaprak

adım 0 / 6



Her değer tek node (düğümlü) bir ağaçtır

En küçük iki olasılık aranır: 0,1 değerli altı yaprak var

16

Huffman ağacının kurulması, her değerin tek düğümlü bir ağaç olarak ele alınmasıyla başlamaktadır. Başlangıçta yedi yaprak bulunmakta, a harfinin olasılığı 0,4, diğer altı harfin olasılığı 0,1'dir. Her adımda olasılığı en küçük iki ağaç seçilecek ve tek bir ağaçta birleştirilecektir. Birleştirme sayısı her zaman yaprak sayısının bir eksiğidir, bu örnekte altı birleştirme yapılacaktır.

ADIM 1 / 6

Huffman ağacı

b + c = 0,2

adım 1 / 6



b ve c birleşir: yeni düğüm 0,2

Eşitlikte soldaki ikisi seçilir

17

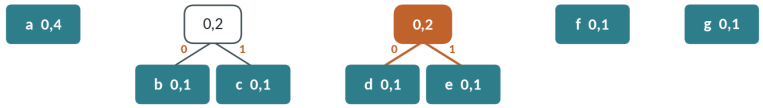
İlk adımda olasılığı 0,1 olan altı yaprak arasından iki tanesinin seçilmesi gerekmektedir. Eşit olasılıklar arasında seçim kararsızdır. Burada soldaki iki yaprak olan b ve c alınmaktadır. İkisi, olasılığı 0,2 olan yeni bir iç düğümün çocukları olmaktadır. Sol dala 0, sağ dala 1 biti atanmaktadır. Farklı bir eşitlik kuralı farklı bir ağaç üretir, ancak ortalama kod uzunluğu değişmez.

ADIM 2 / 6

Huffman ağacı

d + e = 0,2

adım 2 / 6



d ve e birleşir: yeni düğüm 0,2

Listede üç yaprak ve iki alt ağaç kalır

18

İkinci adımda en küçük olasılıklar yine 0,1 değerli yapraklardır. d ve e yaprakları birleştirilmekte ve 0,2 olasılıklı ikinci iç düğüm oluşmaktadır. Listede a yaprağı, iki tane 0,2 değerli alt ağaç ile f ve g yaprakları kalmaktadır.

ADIM 3 / 6

Huffman ağacı

f + g = 0,2

adım 3 / 6



f ve g birleşir: yeni düğüm 0,2

Artık bütün yapraklar bir ağaca bağlı, a hariç

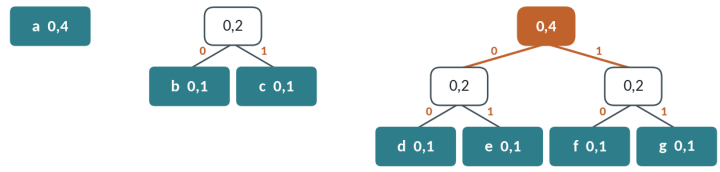
Üçüncü adımda son iki 0,1 değerli yaprak olan f ile g birleştirilmektedir. Bu adımın sonunda listede 0,4 olasılıklı a yaprağı ve üç tane 0,2 olasılıklı alt ağaç bulunmaktadır. Yapraklardan yalnızca a henüz hiçbir birleştirmeye katılmamıştır, çünkü olasılığı en büyük olan değerdir.

ADIM 4 / 6

Huffman ağacı

0,2 + 0,2 = 0,4

adım 4 / 6



de ve fg alt ağaçları birleşir

d, e, f, g köke bir adım daha uzaklaşır

20

Dördüncü adımda en küçük iki olasılık üç tane 0,2 değerli alt ağaç arasından seçilmektedir. Burada de ve fg alt ağaçları birleştirilmekte ve 0,4 olasılıklı bir düğüm oluşmaktadır. Bu birleştirmeyle d, e, f ve g yaprakları köke bir adım daha uzaklaşmakta, dolayısıyla kodlarına birer bit daha eklenecektir.

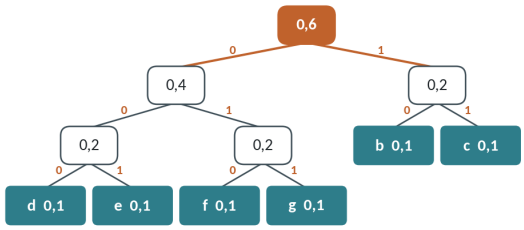
ADIM 5 / 6

Huffman ağacı

0,4 + 0,2 = 0,6

adım 5 / 6

a 0,4



defg ile bc birleşir: 0,6

Kalan: a 0,4 ve 0,6

21

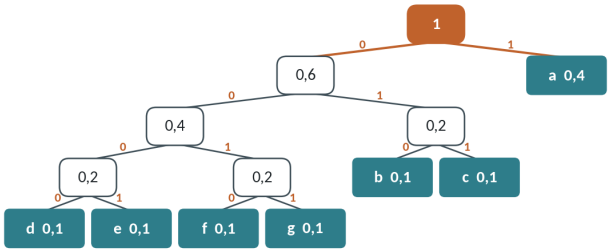
Beşinci adımda listede 0,4 olasılıklı a yaprağı, 0,4 olasılıklı defg alt ağacı ve 0,2 olasılıklı bc alt ağacı bulunmaktadır. En küçük iki değer 0,2 ile 0,4 olduğundan bc alt ağacı ile defg alt ağacı birleştirilmekte ve 0,6 olasılıklı düğüm oluşmaktadır. Eşit olan iki 0,4 değerinden alt ağaç seçilmiştir.

ADIM 6 / 6

Huffman ağacı

0,6 + 0,4 = 1,0

adım 6 / 6



Kök oluşur: ağaç tamam

Kodlar: a 1 · b 010 · c 011 · d 0000

22

Son adımda kalan iki ağaç, 0,6 olasılıklı iç düğüm ile 0,4 olasılıklı a yaprağıdır. İkisi birleştirildiğinde olasılığı 1,0 olan kök oluşmaktadır. Kökten her yaprağa giden yol boyunca okunan bitler kodu vermektedir. a harfi kökün sağ çocuğu olduğundan tek bitlik 1 kodunu, b ve c üç bitlik, d, e, f ve g ise dört bitlik kodları almaktadır.

4

Dictionary-based (sözlük tabanlı) compression (sıkıştırma)

Daha önce görülene geriye başvurmak

KURAL

Ayrıştırma

Her parça, o ana dek görülmemiş en kısa dizge

Sonuç: her yeni parça = görülmüş bir parça + tek karakter

Görülmüş parça yerine sözlükteki numarası yazılır

- Sözlük saklanmaz: kodlarken kurulur, çözerken yeniden üretilir

Parçalar sırayla numaralanır: 1, 2, 3, …

- Sıfır numarası boş dizgeyi gösterir

Kural tek biçimde belirlenmiştir

- Aynı akım her zaman aynı parçalara bölünür

Huffman tek tek değerlerin sıklığından yararlanırken sözlük yöntemleri değer dizilerinin yinelenmesinden yararlanır: daha önce görülmüş bir parça yeniden geçtiğinde parçanın kendisi yerine sözlükteki numarası yazılır. Sözlük dosyada saklanmaz, kodlama sırasında kurulur, çözme sırasında aynı kurallarla yeniden üretilir.

ÖRNEK

Akımın parçalara bölünmesi

aaababbbaaabaaaaaaabaabb

No	1	2	3	4	5	6	7	8	9	10
Parça	a	aa	b	ab	bb	aaa	ba	aaaa	aab	aabb
Önek	–	a	–	a	b	aa	b	aaa	aa	aab
Kod	0a	1a	0b	1b	3b	2a	3a	6a	2b	9b

Kod = önek parçanın No'su + yeni karakter

- bb: önek b, b 3 numaralı parça → 3b
- Önek yoksa 0: a → 0a

Parça uzunlukları ilerledikçe büyür

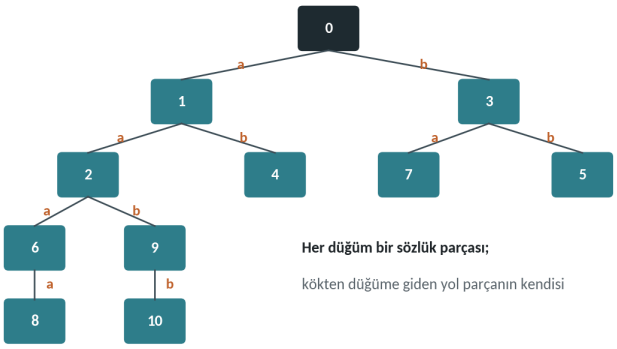
- Uzun dosyalarda compression (sıkıştırma) oranı bu nedenle iyileşir

25

Her adımda akımdan o ana dek görülmemiş en kısa dizge alınır. Bu dizgenin son karakteri dışındaki kısmı, daha önce görülmüş bir parçadır ve önek olarak adlandırılır. Kod, önek parçanın sıra numarası ile yeni karakterden oluşur. Beşinci parça bb'nin öneki b'dir; b üçüncü parça olduğundan bb 3b olarak kodlanır. Onuncu parça aabb'nin öneki dokuzuncu parça aab'dir ve kod 9b olur. Öneki olmayan, yani tek karakterden oluşan parçalarda sıfır numarası boş dizgeyi gösterir: a 0a, b 0b olarak kodlanır. Sözlük genişledikçe daha uzun dizgeler görülmüş sayıldığından parça uzunlukları büyür ve uzun dosyalarda sıkıştırma oranı iyileşir.

YAPI

Sözlük ağacı



Her düğüm bir sözlük parçası;

kökten düğüme giden yol parçanın kendisi

Yeni parça eklemek, var olan bir düğüme tek çocuk eklemektir

Kodlama sırasında kurulan sözlük bir ağaç olarak düşünülebilir. Kökten bir düğüme giden yol boyunca okunan karakterler o düğümün parçasını verir. Düğümün üzerindeki sayı ise parçanın numarasıdır. Yeni bir parça eklenmesi, var olan bir düğüme tek bir çocuk eklemekten ibarettir. Bu yapı, dizginin sözlükte bulunup bulunmadığının karakter karakter ilerlenerek sınanmasını sağlar.

HAREKETLI ŞEKIL

LZ78 parça parça

LZ78 ayrıştırması **1. parça: a**

a	a	a	b	a	b	b	b	a	a	a	b	a	a	a	a	a	a	a	b	a	a	b	b

sözlük

1 a

0a

“a” hiç görülmedi: boş dizge (0) + “a”

Görülmemiş en kısa dizge bir parça olur

Kod: önekin numarası + son karakter

İŞLEM

Çözme

Çözme, kodlamanın tersidir ve aynı sözlüğü yeniden kurar

Her çift okunduğunda

- Numarası verilen parça sözlükten alınır
- Sonuna yeni karakter eklenir
- Elde edilen dizge çıktıya yazılır ve sözlüğe eklenir

Kodlayıcı ile çözücü aynı sözlüğü aynı sırayla üretir

- Sözlüğün dosyada saklanmasına gerek kalmaz

Çözme, kodlamanın tersidir ve aynı sözlüğü yeniden kurar. Her çift okunduğunda, numarası verilen parça sözlükten alınır, sonuna yeni karakter eklenir ve elde edilen dizge hem çıktıya yazılır hem de sözlüğe yeni parça olarak eklenir. Kodlayıcı ile çözücü aynı sözlüğü aynı sırayla ürettiği için sözlüğün dosyada saklanmasına gerek kalmaz. Örneğin 3b çifti okunduğunda sözlüğün üçüncü girdisi olan b dizgesi alınmakta, sonuna b eklenmekte ve bb dizgesi hem çıktıya yazılmakta hem de sözlüğe beşinci girdi olarak eklenmektedir.

5

Kayıpsız sıkıştırma ve doğrudan erişim

Kayıplı sıkıştırma ve kaydın yerinin yitirilmesi

KAVRAM

Geri döndürülemez compression (sıkıştırma)

Bilginin bir bölümünün gözden çıkarılabileceği varsayımı

- 400 × 400 görüntü → 100 × 100: özgün değerler geri hesaplanamaz

Ses, görüntü ve videoda kabul edilebilir

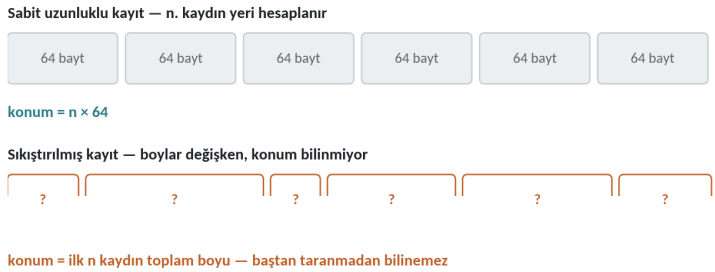
Kayıt, program ve arşiv dosyalarında kullanılmaz

- Tek bir bitin değişmesi anlamı bozar

Geri döndürülemez compression (sıkıştırma), bilginin bir bölümünün gözden çıkarılabileceği varsayımına dayanır. Dört yüze dört yüz piksellik bir görüntünün yüze yüz piksele indirilmesinde her yeni piksel özgün görüntünün on altı pikseline karşılık gelir ve özgün değerler geri hesaplanamaz. Ses, görüntü ve video gibi verilerde yitirilen ayrıntının uygulamada karşılığı yoksa bu yaklaşım kabul edilebilir. Kayıt dosyalarında, program ve arşiv dosyalarında ise tek bir bitin değişmesi anlamı bozduğundan kullanılmaz. Haftanın dosya yapısı açısından ilgilendiren yöntemleri bu nedenle geri döndürülebilir olanlardır.

SONUÇ

Direct access (doğrudan erişimin) yitirilmesi



Dördüncü haftanın belirleyici üstünlüğü ortadan kalkar

Konum ancak baştan tarayarak bulunabilir

Fixed-length record (sabit uzunluklu kayıtlarda) n numaralı kaydın konumu, kayıt boyuyla çarpma yapılarak bulunur. Compression (sıkıştırma) uygulandığında kayıt boyları değişken duruma gelir ve çarpma işlemi anlamını yitirir. Bir kaydın konumu, ancak kendisinden önceki bütün kayıtların boyları toplanarak bulunabilir. Bu da dosyanın baştan taranması demektir. Sıkıştırma ile direct access (doğrudan erişim), olduğu gibi bir arada bulunamaz.

ÇÖZÜM

İki yol

Blok blok compression (sıkıştırma)

- Dosya bütün olarak değil, bloklar hâlinde sıkıştırılır
- Her blok kendi içinde çözülebilir
- Bloğa doğrudan erişilir, yalnızca o blok çözülür

Konumları indeks dosyasında tutmak

- Dördüncü haftadaki indeks yöntemi burada da geçerli

İki yol da onuncu haftanın indeksleme konusuna bağlanır

Uygulamada iki yol izlenir. Birinci yolda dosya bütün olarak değil blok blok sıkıştırılır. Her blok kendi içinde çözülebildiğinden bloğa doğrudan erişilir ve yalnızca o blok çözülür. İkinci yolda kayıt konumları ayrı bir indeks dosyasında tutulur. Dördüncü haftada variable-length record (değişken uzunluklu kayıtlar) için anılan indeks yöntemi burada da geçerlidir.

6

Binary tree method (ikili ağaç yöntemi)

Ağacın bir karar aracı olarak ilk görünümü

NEDEN

Brent'in tek adımını genelleştirmek

- Brent yalnızca birincil zincirdeki kayıtları birer adım taşır
- Bazen iki kaydı art arda taşımak daha kısa toplam verir
- Her düğümde iki seçenek: devam (continue) ya da taşı (move)
  - Seçenekler bir ikili ağaç oluşturur
- Genişlik öncelikli tarama: en az probe veren ilk boş yer
- Ekleme daha da pahalılaşır, başarılı arama daha da kısalır

Brent's metodu collision (çarpışma) anında yalnızca birincil zincir üzerindeki yerleşik kayıtların kendi ikincil zincirlerinde bir adım ilerletilmesini dener. Oysa kimi durumlarda iki kaydın art arda taşınması toplam probe sayısını daha da azaltır. Binary tree method (ikili ağaç yöntemi) her adımda iki seçeneği birlikte değerlendirir: yeni kayıtla yola devam etmek ya da yoldaki yerleşik kaydı taşımak. Bu seçenekler bir ikili ağaç oluşturur. Ağaç genişlik öncelikli dolaşıldığında bulunan ilk boş adres, en az toplam probe sayısını veren çözümdür. Ekleme maliyeti Brent's metodundan da yüksektir; karşılığında başarılı aramanın ortalaması daha da düşer.

FIKIR

Continue ve move

Brent's yönteminde olduğu gibi kayıtlar yer değiştirebilir

Karar bu kez bir ikili ağaç üzerinde verilir

Bir kayıt için iki farklı işlem yapılabilir

- Continue: birincil probe zincirinde bir sonraki adrese gidilir
- Move: birincil zincirdeki kayıt, kendi ikincil zincirinde ilerletilir

Sola dallanma continue, sağa dallanma move

Binary tree method (ikili ağaç yönteminde), Brent's yönteminde olduğu gibi başlangıçta home address (ev adresine) yerleştirilen kayıtlar collision resolution (çarpışma çözümleme) sırasında yer değiştirebilmektedir. Collision (çarpışma) olduğunda kayıtların ne zaman taşınması gerektiği ya da nereye taşınacağı bir ikili ağaç üzerinde belirlenir. Bir kayıt için iki farklı işlem yapılabilir. Continue işleminde birincil probe zincirinde bir sonraki adrese gidilir. Move işleminde ise birincil probe zincirindeki kayıt, kendi ikincil probe zincirinde bir sonraki adrese taşınır.

YAPI

Ağaç nasıl kurulur

Yukarıdan aşağıya ve soldan sağa: genişlik öncelikli

Her düğüm bir adres ve o adrese yerleştirilmeye çalışılan anahtar taşır

Sol çocuğun adresi

- geçerli adres + taşınmakta olan anahtarın ikinci hash değeri

Sağ çocuğun adresi

- geçerli adres + o adreste duran kaydın ikinci hash değeri

Her düğüm için iki seçim vardır: sola dallanma continue, sağa dallanma move işleminde kullanılır. Ağaç yukarıdan aşağıya ve soldan sağa, yani genişlik öncelikli kurulur. Her düğüm bir adres ile o adrese yerleştirilmeye çalışılan anahtarı taşır. Sol çocuğun adresi, geçerli adrese taşınmakta olan anahtarın ikinci hash değeri eklenerek, sağ çocuğun adresi ise geçerli adreste duran kaydın ikinci hash değeri eklenerek bulunur.

ALGORITMA

Kayıt ekleme

Anahtara birinci hash function (hash fonksiyonunu) uygula

Home address (ev adresi) boşsa kaydı buraya ekle, değilse

- Boş bir yaprak ya da "tablo dolu" çıkana dek ağacı genişlet

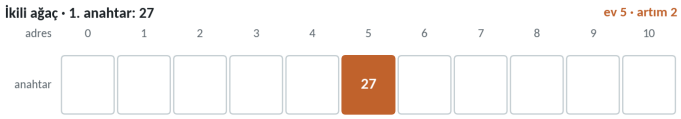
Boş yaprak bulununca kökten yaprağa uzanan yolu geriye yürü

- Her sağ dal bir taşımayı gerçekleştirir
- Ebeveynin anahtarı ebeveynin adresine yazılır
- Sol dallarda bir şey yapılmaz: aynı anahtar ilerlemiştir

Eklenecek kaydın anahtarına birinci hash function (hash fonksiyonu) uygulanır. Home address (ev adresi) boşsa kayıt buraya eklenir. Değilse, boş bir alan ya da tablo dolu iletisiyle karşılaşılıncaya kadar ağaç genişletilir. Ağaç üzerinde boş bir yaprak bulunduğunda algoritma sonlandırılır. Ardından kökten yaprağa uzanan yol geriye doğru yürünür: her sağ dal bir taşımayı gerçekleştirir, ebeveynin anahtarı ebeveynin adresine yazılır. Sol dallarda bir şey yapılmaz, çünkü aynı anahtar ilerlemiştir.

ADIM 1 / 10

İkili ağaç: 27



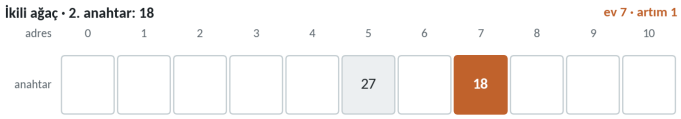
27 mod 11 = 5: home address (ev adresi) boş

38

Birinci adımda 27 anahtarı eklenmektedir. Home address (ev adresi) 5, artımı 2 değeridir. Ev adresi boş olduğundan kayıt doğrudan yerleşmekte ve ağaç kurulmamaktadır. Bu adımdan sonra tabloda bir kayıt bulunmaktadır.

ADIM 2 / 10

İkili ağaç: 18



18 mod 11 = 7: home address (ev adresi) boş

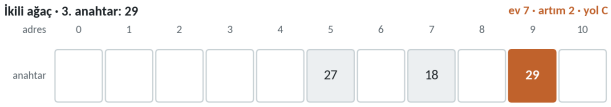
39

İkinci adımda 18 anahtarı eklenmektedir. Home address (ev adresi) 7, artımı 1 değeridir. Ev adresi boş olduğundan kayıt doğrudan yerleşmekte ve ağaç kurulmamaktadır. Bu adımdan sonra tabloda iki kayıt bulunmaktadır.

ADIM 3 / 10

İkili ağaç: 29





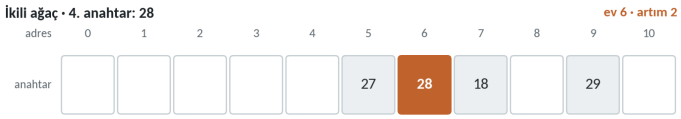
Yol C · 2 düğüm · taşıma yok, 29 → 9

40

Üçüncü adımda 29 anahtarı eklenmektedir. Home address (ev adresi) 7, artımı 2 değeridir. Ev adresi doludur ve ağaç genişlik öncelikli kurulmaktadır. Sol dal continue, yani aynı anahtarın kendi artımıyla ilerlemesi, sağ dal move, yani o adresteki kaydın kendi artımıyla ilerlemesidir. İlk boş adres iki düğüm üretildikten sonra bulunmaktadır ve yol C biçimindedir. Yolda move dalı bulunmadığından hiçbir kayıt taşınmamakta ve 29 doğrudan 9 adresine yazılmaktadır.

ADIM 4 / 10

İkili ağaç: 28



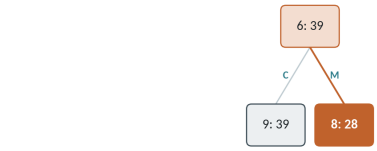
28 mod 11 = 6: home address (ev adresi) boş

41

Dörtüncü adımda 28 anahtarı eklenmektedir. Home address (ev adresi) 6, artımı 2 değeridir. Ev adresi boş olduğundan kayıt doğrudan yerleşmekte ve ağaç kurulmamaktadır. Bu adımdan sonra tabloda dört kayıt bulunmaktadır.

ADIM 5 / 10

İkili ağaç: 39





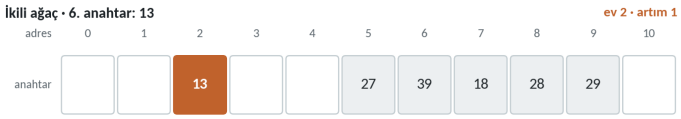
Yol M · 3 düğüm · taşınan 28 → 8

42

Beşinci adımda 39 anahtarı eklenmektedir. Home address (ev adresi) 6, artımı 3 değeridir. Ev adresi doludur ve ağaç genişlik öncelikli kurulmaktadır. Sol dal continue, yani aynı anahtarın kendi artımıyla ilerlemesi, sağ dal move, yani o adresteki kaydın kendi artımıyla ilerlemesidir. İlk boş adres üç düğüm üretildikten sonra bulunmaktadır ve yol M biçimindedir. Geriye yürünürken her move dalı bir taşıma gerçekleştirmektedir: 28 → 8. Yeni kayıt 39, boşalan adrese, 6 numaralı adrese yazılmaktadır.

ADIM 6 / 10

İkili ağaç: 13



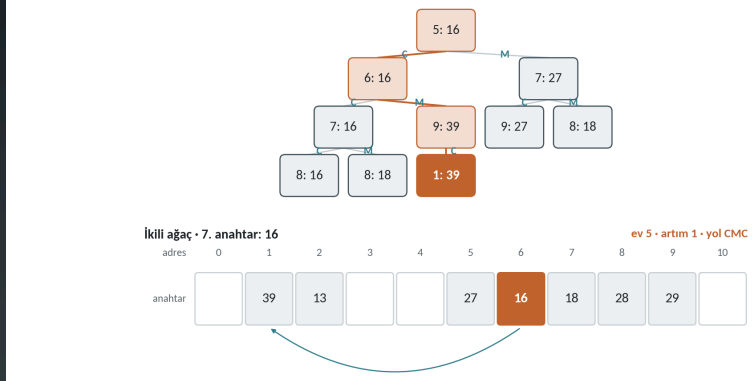
13 mod 11 = 2: home address (ev adresi) boş

43

Altıncı adımda 13 anahtarı eklenmektedir. Home address (ev adresi) 2, artımı 1 değeridir. Ev adresi boş olduğundan kayıt doğrudan yerleşmekte ve ağaç kurulmamaktadır. Bu adımdan sonra tabloda altı kayıt bulunmaktadır.

ADIM 7 / 10

İkili ağaç: 16



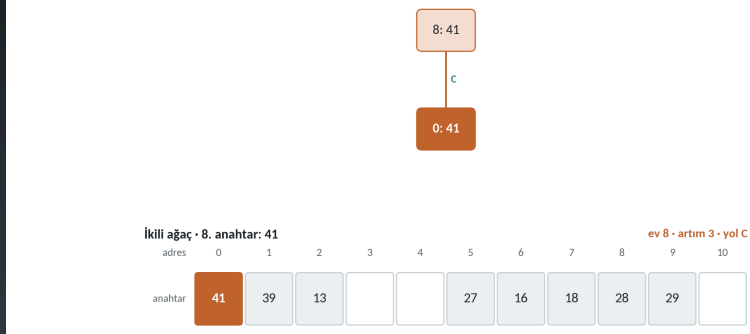
Yol CMC · 10 düğüm · taşınan 39 → 1

44

Yedinci adımda 16 anahtarı eklenmektedir. Home address (ev adresi) 5, artımı 1 değeridir. Ev adresi doludur ve ağaç genişlik öncelikli kurulmaktadır. Sol dal continue, yani aynı anahtarın kendi artımıyla ilerlemesi, sağ dal move, yani o adresteki kaydın kendi artımıyla ilerlemesidir. İlk boş adres on düğüm üretildikten sonra bulunmaktadır ve yol CMC biçimindedir. Geriye yürünürken her move dalı bir taşıma gerçekleştirmektedir: 39 → 1. Yeni kayıt 16, boşalan adrese, 6 numaralı adrese yazılmaktadır.

ADIM 8 / 10

İkili ağaç: 41



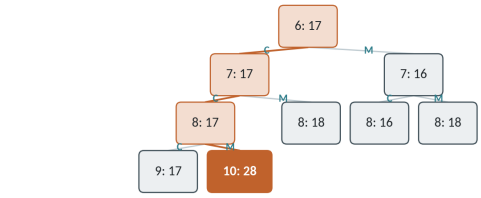
Yol C · 2 düğüm · taşıma yok, 41 → 0

45

Sekizinci adımda 41 anahtarı eklenmektedir. Home address (ev adresi) 8, artımı 3 değeridir. Ev adresi doludur ve ağaç genişlik öncelikli kurulmaktadır. Sol dal continue, yani aynı anahtarın kendi artımıyla ilerlemesi, sağ dal move, yani o adresteki kaydın kendi artımıyla ilerlemesidir. İlk boş adres iki düğüm üretildikten sonra bulunmaktadır ve yol C biçimindedir. Yolda move dalı bulunmadığından hiçbir kayıt taşınmamakta ve 41 doğrudan 0 adresine yazılmaktadır.

ADIM 9 / 10

İkili ağaç: 17



İkili ağaç · 9. anahtar: 17 — ev 6 · artım 1 · yol CCM

adres	0	1	2	3	4	5	6	7	8	9	10
anahtar	41	39	13			27	16	18	17	29	28

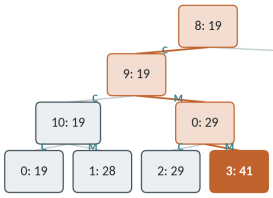
Yol CCM · 9 düğüm · taşınan 28 → 10

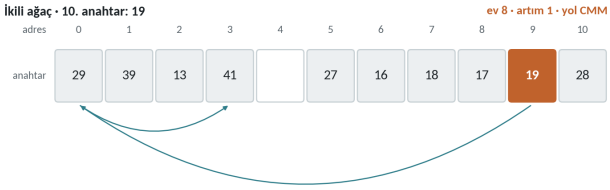
46

Dokuzuncu adımda 17 anahtarı eklenmektedir. Home address (ev adresi) 6, artımı 1 değeridir. Ev adresi doludur ve ağaç genişlik öncelikli kurulmaktadır. Sol dal continue, yani aynı anahtarın kendi artımıyla ilerlemesi, sağ dal move, yani o adresteki kaydın kendi artımıyla ilerlemesidir. İlk boş adres dokuz düğüm üretildikten sonra bulunmaktadır ve yol CCM biçimindedir. Geriye yürünürken her move dalı bir taşıma gerçekleştirmektedir: 28 → 10. Yeni kayıt 17, boşalan adrese, 8 numaralı adrese yazılmaktadır.

ADIM 10 / 10

İkili ağaç: 19





Yol CMM · 11 düğüm · taşınan 41 → 3, 29 → 0

47

Onuncu adımda 19 anahtarı eklenmektedir. Home address (ev adresi) 8, artımı 1 değeridir. Ev adresi doludur ve ağaç genişlik öncelikli kurulmaktadır. Sol dal continue, yani aynı anahtarın kendi artımıyla ilerlemesi, sağ dal move, yani o adresteki kaydın kendi artımıyla ilerlemesidir. İlk boş adres on bir düğüm üretildikten sonra bulunmaktadır ve yol CMM biçimindedir. Geriye yürünürken her move dalı bir taşıma gerçekleştirmektedir: 41 → 3, 29 → 0. Yeni kayıt 19, boşalan adrese, 9 numaralı adrese yazılmaktadır.

ÖRNEK

On anahtarın sonucu

$H_1(k) = k \bmod 11$ $H_2(k) = \text{bölüm}(k / 11)$ · **27, 18, 29, 28, 39, 13, 16, 41, 17, 19**

Adres	0	1	2	3	4	5	6	7	8	9	10
Kayıt	29	39	13	41	:	27	16	18	17	19	28
Probe	3	3	1	3	:	1	2	1	3	2	3

toplam 22 probe · ortalama 2,20

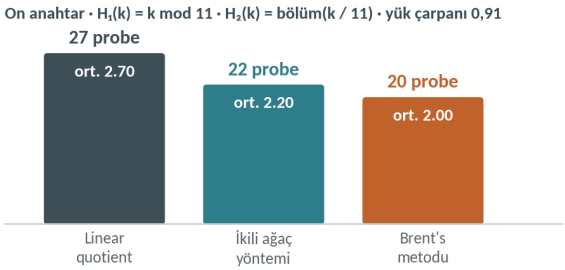
elde edilen düzen ÖA'nın çözümlü örneğiyle birebir aynı

Kayıt ekleme için ikili ağaç, kayıt erişimi için linear quotient kullanılır

Taşınan kayıtlar da kendi home address (ev adreslerinden) başlayan zincirde bulunabilir kalır

KARŞILAŞTIRMA

Üç yöntem, aynı anahtarlar



Literatürdeki genel sonuç: yüksek yükte ikili ağaç ortalamada Brent's'in altına iner

Tek bir on anahtarlık örnek bu ortalamayı göstermez: burada Brent's önde

ALIŞTIRMA 1 · SORU

Binary tree method (ikili ağaç yöntemi)

$H_1(k) = k \bmod 10$ $H_2(k) =$ bölüm$(k / 10)$ · 15, 23, 80, 53, 30, 43, 60, 32, 73

Anahtar	Ev adresi	Artım	Ağaç yolu (C / M)	Taşınan	Yerleşti
15	?	?	?	?	?
23	?	?	?	?	?
80	?	?	?	?	?
53	?	?	?	?	?
30	?	?	?	?	?
43	?	?	?	?	?
60	?	?	?	?	?
32	?	?	?	?	?
73	?	?	?	?	?

Her collision (çarpışmada) ağacı çizin, düğümleri genişlik öncelikli numaralandırın

Aynı anahtarlarla linear quotient tablosuyla karşılaştırın

ALIŞTIRMA 1 · ÇÖZÜM

Binary tree method (ikili ağaç yöntemi)

Anahtar	Ev	Artım	Yol	Taşınan	Yerleşti	Probe
15	5	1	ev boş		5	1
23	3	2	ev boş		3	1
80	0	8	ev boş		0	1
53	3	5	C		8	2
30	0	3	CC		6	3
43	3	4	C		1	3
60	0	6	CC		4	5
32	2	3	MC	60 → 4	2	1
73	3	7	CCM	43 → 1	7	3

Toplam 20 probe, ortalama 2,22

32 ve 73 eklenirken yerleşik kayıtlar taşınır

7

Uygulama

Binary tree method (ikili ağaç yöntemi) C++ gerçeklemesi

C++ · 1

Binary tree method (ikili ağaç yöntemi) · ekleme

```
constexpr int BOS = -1;
struct Dugum { int adres, anahtar, ebeveyn; bool tasima; };

bool agac_ekle(vector<int>& t, int k, int (*h2)(int)) {
    const int m = t.size();
    vector<Dugum> d{{k % m, k, -1, false}};
    for (size_t i = 0; i < d.size() && d.size() < 4u * m; ++i) {
        Dugum x = d[i];
        if (t[x.adres] == BOS) {
            t[x.adres] = x.anahtar;
            for (int j = i; d[j].ebeveyn != -1; j = d[j].ebeveyn)
                if (d[j].tasima) {
                    const Dugum& e = d[d[j].ebeveyn];
                    t[e.adres] = e.anahtar;
                }
            return true;
        }
        d.push_back({(x.adres + h2(x.anahtar)) % m, x.anahtar, int(i), false});
        d.push_back({(x.adres + h2(t[x.adres])) % m, t[x.adres], int(i), true});
    }
    return false;
}
```

Ağaç açıkça tutulmaz: düğüm vektörü ve ebeveyn indisi

ÇÖZÜMLÜ ÖRNEK

İkili ağaç, on anahtar

```
int h2(int k) { return k / 11; }
vector<int> t(11, BOS);
for (int k : {27, 18, 29, 28, 39, 13, 16, 41, 17, 19}) agac_ekle(t, k, h2);
for (int a = 0; a < 11; ++a) {
    if (t[a] == BOS) continue;
    int k = t[a], p = k % 11, n = 1;
    while (p != a) { p = (p + h2(k)) % 11; ++n; }
    toplam += n;
    cout << a << ':' << k << '(' << n << ')' << (++say % 5 ? "  " : "\n");
}
```

ÇIKTI

```
0:29(3)  1:39(3)  2:13(1)  3:41(3)  5:27(1)
6:16(2)  7:18(1)  8:17(3)  9:19(2)  10:28(3)
toplam 22, ortalama 2.2
```

22 probe: ÖA'nın tablosuyla aynı

Ekleme ağaçla, erişim linear quotient ile